COURSE ASSIGNMENT / DOMAIN MAP

课程大作业 · 领域地图专题

零基础讲解版

AI 模型构建
领域地图与学习路径

从问题定义到可复现的模型系统

知识 · 方法 · 工具 · 课程 · 实践

本册回答六个问题

01 **这个领域解决什么问题？**
典型任务如何完成

02 **核心概念与原理是什么？**
它们分别解释什么

03 **有哪些技术路线？**
依赖、并列、替代还是协同

04 **软件与模型如何支撑？**
各自承担什么作用

05 **需要什么数据与算力？**
如何评测与复现

06 **课程如何组织？**
如何形成能力与作品



一条主线 · 七个阶段 · 三类证据：可复现、可评测、可治理

> “模型构建不是选一个更大的模型，
> 而是在约束下做出可解释、可复现、可评测的取舍。”

主题

AI 模型构建（AI Model Construction）

范围

问题与任务 / 概念与原理 / 方法路线 / 软件模型 / 数据算力 / 课程路径

资料截止 2026-10-22 · v1.0 · 主要来源见末页

00 / 先读这一页：模型构建到底在干什么

如果你完全没接触过 AI，这一页就是入口。

先不要想“神经网络”“Transformer”这些词。你可以把 **AI 模型构建** 想成开一家餐厅：

- **问题定义**：今天到底卖什么菜？是快餐、火锅，还是只做外卖？
- **数据工程**：买菜、洗菜、切菜。食材不新鲜，厨师再厉害也做不出好菜。
- **模型设计**：定菜谱。是简单的家常菜，还是复杂的宴席？
- **训练优化**：反复练手。同一道菜做一百遍，越做越稳定。
- **对齐后训练**：按客人的口味调整。有人怕辣，有人要少盐。
- **评测验证**：试吃打分。不能只看厨师自己说“好吃”。
- **部署迭代**：开张营业，听顾客反馈，再改菜谱。



图 1 零基础版领域地图：把七个阶段想成开一家餐厅

来源：作者绘制；阶段划分综合 [C01]、[C02]、[B08]。

所以，**模型不是一个“存答案的数据库”，而是一套“从数据里找规律、再拿来做判断或生成”的系统**。构建这套系统，既需要会写代码，也需要会看数据、会做评测、会判断什么时候该换方法。

三种“做模型”的深度

第一种，调用/适配。 就像买半成品菜，回家加热、调味。你用现成的大模型，通过提示词、检索、工具调用或轻量微调，让它完成你的任务。多数应用团队在这一层。

第二种，微调/对齐。 就像把半成品菜按自家口味再加工。你用自己的领域数据，教模型“我们的说法、格式、规矩”。需要几千到几万条好样本。

第三种，预训练/从零。 就像从种菜、养猪开始，做一桌完整的宴席。你要处理海量数据、设计模型、做分布式训练。成本最高，通常是研究机构、头部企业或教学实验。

> **一句话记住**
>
> 模型构建的本质，是在数据、算力、时延、成本、合规与安全的约束下，做出**可复现、可评测、可部署**的取舍。模型只是其中一环。

知识树：先看森林，再看树木

这张图就是全文的“骨架”。先看五个分支，再回到正文看每个叶子的解释。

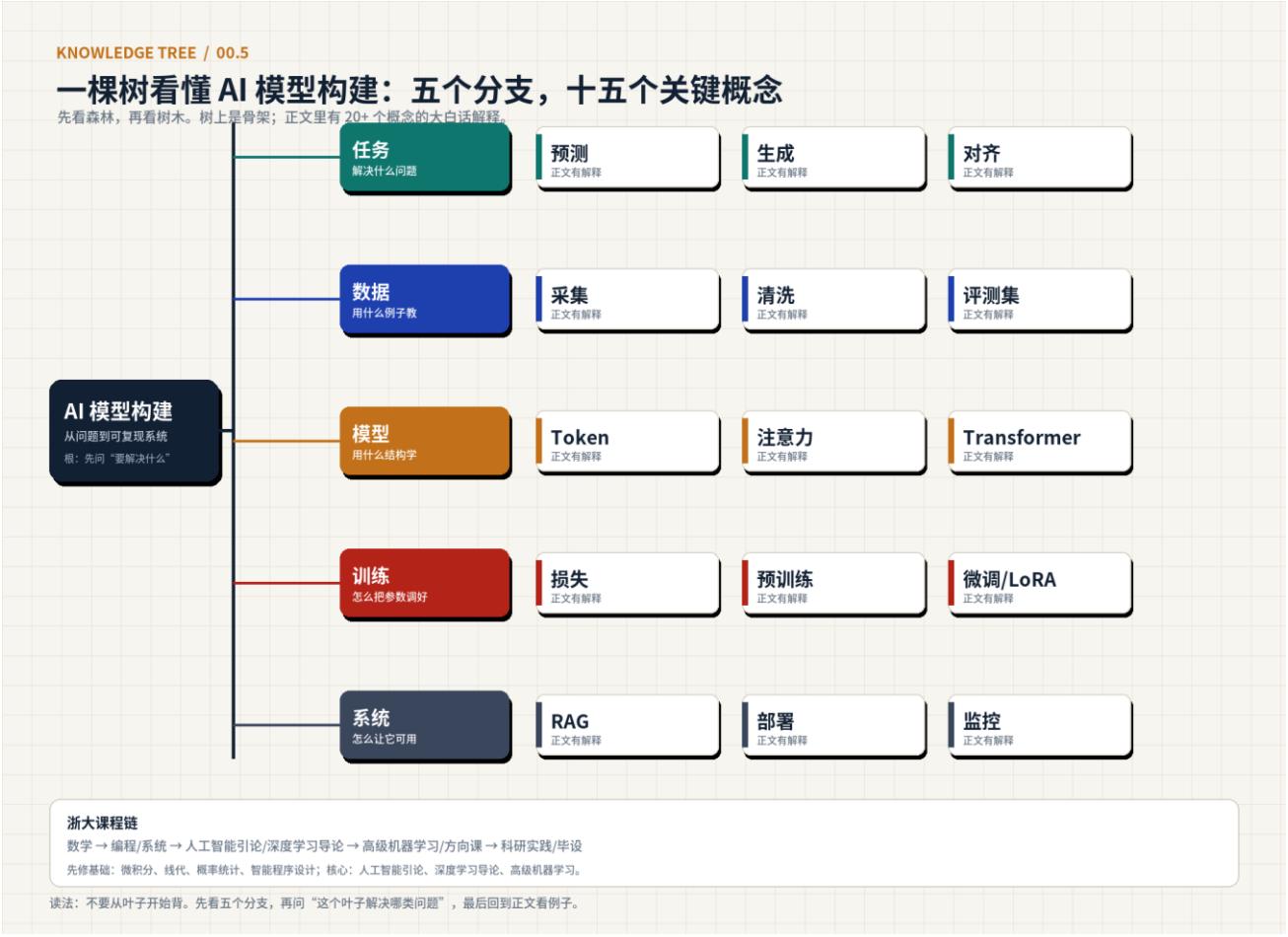


图 2 AI 模型构建知识树：任务、数据、模型、训练、系统五个分支

来源：作者绘制；概念划分综合 [C01]、[C02]、[B08]、[Z01]。

怎么读这棵树？

1. **先看根**：所有分支都服务于同一个问题——把问题变成可复现、可评测、可部署的模型系统。
2. **再看五个分支**：任务（解决什么）、数据（用什么教）、模型（用什么结构）、训练（怎么调好）、系统（怎么用起来）。
3. **最后看叶子**：每个叶子都是一个概念；正文里都有“一句话 + 打个比方 + 什么时候会踩坑”。不要从叶子开始背。

> **这棵树和浙大课程的关系**
>
> 左侧的五个分支，对应浙大培养方案里的“数理基础→编程/系统→AI 核心→方向课→科研实践”。你学的每一门课，最终都要落到某个分支上；否则就只是“上过课”，没有形成能力。

01 / 模型到底在学什么：九个最基础的概念

这一节不写公式。每个概念都用“它在干什么”来解释。

1.1 参数：模型身上数不清的“旋钮”

一句话：参数就是模型里可以被调整的数字。

你可以把模型想成一台有成千上万个旋钮的机器。输入一张图、一句话，旋钮的不同位置会给出不同输出。所谓“学习”，就是拿很多例子，反复微调这些旋钮，让输出越来越接近正确答案。

所以“训练一个模型”不是把知识复制进去，而是**通过例子调整旋钮**。参数越多，旋钮越多，模型能表达的关系越复杂；但也更容易“记过头”，更费显存和算力。

1.2 张量：把数字排成“多维表格”

一句话：张量就是多维数组，是 GPU 最擅长处理的数据形状。

一个数字是标量，一排数字是向量，一张表是矩阵；再往上叠，就是张量。比如一张彩色图片可以看成“高 × 宽 × 颜色通道”的三维张量；一批图片就是四维。

为什么提它？因为模型的计算，本质上是大量矩阵/张量乘法。GPU 就像很多把算盘同时拨，特别适合这种“批量、重复”的计算。你不需要手算张量，但要知道：**模型的速度和显存，和张量的大小直接相关。**

1.3 Token：模型眼里的“字块”

一句话：模型不直接读汉字或单词，它读的是一块一块的 token。

你可以把 token 想成乐高积木。英文单词可能是一块积木，也可能被拆成几块；中文常常一个字或两个字一块。不同的 tokenizer（分词器）会把同一句话切成不同的块。

这带来两个实际后果：第一，**上下文长度**是按 token 算的，不是按字数；第二，API 收费也常按 token 算。中文有时比英文更“费 token”，因为一个汉字可能被拆成多个块。

1.4 嵌入：给每个词一个“坐标”

一句话：嵌入就是把文字、图片等对象，变成一串有意义的数字。

想象一个巨大的地图：意思相近的词，坐标靠得近；“猫”和“狗”比“猫”和“汽车”更近。模型用这种“坐标”来判断相似、做检索、做推荐。

嵌入不是“人工规定的”，而是模型从大量数据里学出来的。所以它也会带偏见：如果数据里“护士”总跟女性一起出现，嵌入也可能把这种偏见带进去。

1.5 模型：一个巨大的“函数”

一句话：模型就是一个从输入到输出的函数，网络结构决定这个函数的“形状”。

你给它一张图，它输出“猫”；你给它一句话，它输出下一个词。中间的“网络结构”（几层、多宽、怎么连接）决定了它擅长处理什么。

但要注意：**模型不是大脑的复制品**。它更像一个被数据“压”出来的统计机器：见过很多类似模式，所以能做出合理猜测；但它没有真正的理解，也不天然知道对错。

1.6 损失：模型每次答题的“扣分”

一句话：损失是一个数字，告诉模型“这次错得有多离谱”。

模型答完题，系统会拿它的输出和正确答案比较，算出一个“扣分”。扣分越大，说明这次越离谱。训练的目标，就是让这个扣分越来越小。

这里有个关键：**损失低，不等于任务表现好。** 如果考试题泄露了、数据重复了，模型可能“背答案”拿到低损失，但换一批题就不行。

1.7 梯度与反向传播：扣分后，谁该负责?

一句话：梯度告诉每个参数“你该往哪个方向调、调多少”；反向传播是把责任从输出一层层传回去。

继续用考试打比方：整张卷子扣了 10 分，不能平均怪每一道题。反向传播会算出：哪个参数对这次错误“贡献大”，哪个贡献小。贡献大的多调一点，贡献小的少调一点。

没有反向传播，深度学习就没法训练。你不需要手推公式，但要知道：**梯度可能消失、爆炸，也可能被脏数据带偏。**

1.8 优化器与学习率：下山时每一步走多大

一句话：优化器决定“怎么走”，学习率决定“每一步走多大”。

把训练想成下山：山底是损失最小的地方。学习率太大，一步跨过山谷，来回震荡；太小，走得很慢，甚至卡在半山腰。优化器（如 AdamW）会根据历史梯度，自动调整每个参数的方向和步长。

所以调学习率不是玄学：它是在“快”和“稳”之间找平衡。 warmup（先小步热身）、余弦衰减（越接近终点步子越小）都是常见做法。

1.9 Epoch、Batch 与过拟合：刷题刷多少遍，才算真会?

一句话：Epoch 是把题库刷几遍，Batch 是每次拿多少题；过拟合是把练习册答案背下来了。

“Batch”是一次喂给模型的一小批样本；“Epoch”是把整个训练集完整过一遍。批太小，训练抖动；批太大，显存吃不消，也不一定更好。

“过拟合”是模型在训练集上分数很高，换新题就崩。就像学生把练习册答案背得滚瓜烂熟，但没理解方法。解决办法不是“再刷一遍”，而是更多样的数据、正则化、早停，以及**独立的验证集**。

把九个概念串起来： 数据被切成 token，token 变成嵌入，嵌入送进模型；模型给出预测，损失算出“扣了多少分”，梯度算出“谁该负责”，优化器负责“怎么调旋钮”。训练就是把这件事重复很多遍，直到模型在**没见过的题**上也能答对。

02 / 模型怎么“看懂”一句话：九个进阶概念

这一节解释大模型时代最常听到的词。仍然不写公式。

2.1 注意力：读一句话时，眼睛该看哪里

一句话：注意力让模型在处理某个词时，自动去看句子里跟它最相关的其他词。

比如“小明把书放在桌子上，因为它太重了”——“它”指书还是桌子？人会自动看上下文；注意力就是让模型也学会“看哪里”。

你可以把它想成查字典：当前这个词发出一个“我想找什么”的查询，其他词给出“我有什么”的标签，两边匹配度高的，就多取一点信息。这就是 Q（查询）、K（键）、V（值）的直觉，不需要背公式。

2.2 Transformer：用“注意力”搭起来的积木塔

一句话：Transformer 是一种把“注意力”作为核心积木的模型结构。

它不像以前的模型那样一个字一个字往后读，而是可以一次看到整句话，并行计算。这让它特别适合 GPU，也特别适合长文本。今天的大语言模型，绝大多数都是 Transformer 的变体。

但 Transformer 不是万能的：文本越长，注意力计算越贵；参数越大，显存越高。小数据、边缘设备上，简单的 CNN 或树模型可能更划算。

2.3 预训练：先海量阅读，再学专业课

一句话：预训练就是让模型先读海量通用数据，学会语言和世界常识。

这就像一个人先读很多书、看很多新闻，再去做具体工作。预训练通常用“自监督”任务：给一段文本，遮住一部分让模型猜，或者让它预测下一个词。**自监督就是“数据自己出题”**，不需要人工标注，所以能利用互联网级别的数据。

预训练很贵：数据、算力、工程缺一不可。小团队通常不需要从零预训练，直接用开源/商用基座更现实。

2.4 微调：入职培训

一句话：微调就是拿自己的任务数据，让预训练模型“再学一下我们这儿的规矩”。

比如你希望模型用公司的术语、按固定的表格回答、只处理某类工单。预训练模型已经有通用能力，微调只是把它“掰”向你的场景。

微调不是“重新训练一遍”。数据质量、学习率、训练轮数都很关键；数据太单一，模型会“学傻”，通用能力反而下降。

2.5 LoRA：不改原书，贴一张便签

一句话：LoRA 是只训练一小块“补丁”，不动原来模型的大部分参数。

原模型像一本很厚的书，你不想重印，只在某些页贴便签，告诉它“遇到这种问题，按便签上的方式处理”。便签很小，所以单卡也能训；换任务时，换一张便签就行。

LoRA 的局限是容量有限：它能改风格、格式、部分行为，但不适合灌入大量新知识。知识型问题，优先考虑 RAG。

2.6 对齐：教它“好好说话、别乱说、不懂就拒答”

一句话：对齐就是让模型的行为符合人的偏好和规范。

模型会生成，不等于它会“好好生成”。对齐要教它：有用、诚实、无害；该拒答时拒答，该澄清时澄清。常见方法有 RLHF（用人类偏好训练奖励模型，再用强化学习优化）和 DPO（直接用偏好对优化，省掉奖励模型）。

对齐最难的是“度”：过度安全，会变得什么都不敢答；过度讨好，会顺着用户胡说。所以需要红队、评测和人工复核。

2.7 RAG：开卷考试

一句话：RAG 是“先查资料，再回答”，把知识放在模型外面。

模型像一个闭卷考生，知识有截止时间，也会忘。RAG 给它配一个资料库：先检索相关文档，再把文档片段放进上下文，让它带着证据回答。

RAG 的关键不在“生成”，而在“检索”：切块是否合理、召回是否够、重排是否准、引用是否对。检索错了，后面生成再漂亮也没用。

2.8 量化与蒸馏：把大模型装进小盒子

一句话：量化是“用更少的比特存数字”，蒸馏是“让大模型教小模型”。

量化把 16 位、32 位的权重压成 8 位、4 位，显存和带宽下降，推理更快；代价是可能掉一点精度。蒸馏则是让一个小模型模仿大模型的输出，把能力“迁移”过去。

两者常一起用：先蒸馏得到一个更小的模型，再量化，最后部署到边缘设备。

2.9 幻觉、评测与缩放：它可能在“编”，所以要留证据

一句话：幻觉是模型生成看似合理但没有依据的内容；评测是用独立数据检查它到底行不行；缩放规律是“模型/数据/算力一起加，效果按规律提升，但会递减”。

模型不是数据库，它会“顺着语言概率”编。解决幻觉不能只靠“让它别编”，而要靠引用、拒答、检索、评测和人工兜底。

评测就像模拟考：MMLU、GSM8K、HumanEval 是常见卷子，但可能泄题、押题、与真实任务脱节。所以除了公开基准，还要有**你自己的任务测试集**。

缩放规律告诉你：加大模型/数据通常有用，但边际收益递减，而且推理成本会上升。另一个常听到的词是 **MoE（混合专家）**：一个公司有很多专家，每次只叫最相关的几个来干活，总参数很大，但每次计算量没那么大。

2.10 多模态、扩散模型与智能体

多模态：模型不只看文字，还看图、听声音、看视频。关键不是“接一个图像编码器”，而是让图文/音文在同一个语义空间里对齐。

扩散模型：先把一张图逐渐加噪，再学着从噪声一步步去噪。今天的图像生成，很多都用这类方法。

智能体（Agent）：模型不只回答，还会规划、调用工具、执行多步任务。它更像一个“会使用工具的实习生”，需要权限、回滚、日志和评测，不能当成魔法。

把这一节串起来： 预训练让模型“读过很多书”，微调让它“熟悉你的业务”，LoRA 让这件事更便宜；对齐教它“好好说话”，RAG 让它“开卷考试”，量化和蒸馏让它“跑得动”。这些词不是互斥的，而是同一套系统里不同位置的零件。

03 / 模型能做什么：五类任务与一个完整例子

这一节把概念落到“任务”上。

3.1 五类任务，用生活例子解释

预测：给输入，判断一个结果。比如看一张 X 光片判断有没有病灶，看一条评论判断好评差评。它的输出是标签、数值或位置。

生成：给条件，产生新内容。比如把中文翻译成英文，把一句话扩写成一段文案，根据文字生成图片。它的输出是文本、图像、音频或代码。

表征：把对象变成向量，用来比较、检索、推荐。比如“找相似文章”“给用户推荐商品”“从知识库召回相关段落”。

决策：在环境里选动作，追求长期回报。比如机器人抓取、游戏、仓库调度。它关心的不是“这一步对不对”，而是“长期下来好不好”。

对齐：让模型的行为符合人的偏好。比如该拒答时拒答、该引用时引用、该澄清时澄清。它不是单独的任务，而是贯穿所有任务的要求。

3.2 一个任务从头到尾：企业文档问答

假设你在一家制造企业，有 2,000 页设备手册，工程师想用自然语言问“这个故障码怎么排查”。

1. **写任务卡**：谁用、问什么、答什么、什么算成功、不能做什么。
2. **准备数据**：PDF 解析保留页码/章节；切块；收集 300—500 个真实问题；构造“手册里没有答案”的负样本。
3. **做基线**：先做 BM25+模板，再做闭卷大模型，最后做 RAG。
4. **选路线**：知识型错误优先改检索；格式/语气问题再考虑 LoRA。
5. **评测**：检索召回、答案正确率、引用准确率、拒答率、P95 时延。
6. **上线**：vLLM/内部服务、缓存、日志；每周把失败案例归类，回到数据/检索/提示/微调。

> **任务卡不是形式**
>
> 当“引用准确率 ≥ 90%”和“P95 ≤ 4 秒”冲突时，你才会认真考虑切块、重排、缓存和并行。任务卡的价值，是逼出取舍。

为什么先做基线？ 因为你需要知道“不训练模型能做到什么”。如果 BM25 已经能回答 70% 的问题，你花两周微调只提升 3%，就不划算。基线不是浪费时间，它是判断“值不值得做”的尺子。

04 / 什么时候用哪种方法：把路线当成“选择题”

这一节不堆术语，只讲“为什么选它”。

4.1 先问四个问题

第一，你的数据长什么样？ 表格数据、几千条样本，先用梯度提升树/线性模型（一类在表格数据上很强的传统模型），往往比深度学习更稳。图像、文本、语音，才轮到深度学习。

第二，你有没有标注数据？ 没有标注、但有大量无标注文本，可以先用预训练模型 + 提示；有几千条领域样本，再考虑微调。

第三，知识是不是经常变？ 经常变、需要引用，优先 RAG；风格/格式/行为不对，才考虑微调或对齐。

第四，要不要“动手做事”？ 只是问答，不需要 Agent；要查数据库、发邮件、执行代码，才需要工具调用/智能体，而且必须加权限与回滚。

4.2 六条路线，各自的“适用场景”

经典机器学习：表格、小数据、要可解释。比如信用评分、设备故障预测。它训练快、调试容易，别瞧不起。

深度学习端到端：图像、语音、文本，数据足够。它自己学特征，但需要 GPU 和调参经验。

预训练 + 微调：有通用底座，再用自己的数据“入职培训”。这是目前最常见的模型构建方式。

提示/上下文学习：不改参数，只改输入。适合快速验证、任务多样、标注少；缺点是受上下文长度限制，提示敏感。

RAG：知识型任务。先检索、再生成，能引用、能更新；缺点是检索决定上限，多跳推理弱。

对齐/智能体：前者让模型“好好说话”，后者让模型“会做事”。两者都不是免费午餐：一个需要偏好数据，一个需要工程可靠性。

4.3 它们不是互斥的

RAG 和微调可以一起用：RAG 管“知道什么”，微调管“怎么说”。LoRA 和全参微调是替代关系：资源少用 LoRA，资源足、要最大容量用全参。蒸馏和量化是协同：先把能力迁到小模型，再压小部署。

> **三条硬规则**
>
> 先做不训练的基线；评测集先于模型；记录数据版本与随机种子。违反这三条，后面的“优化”都不可信。

05 / 工具为什么存在：用厨房设备来理解

这一节解释“每个工具解决什么问题”，而不是背工具名。

5.1 六个工具，六个“厨房角色”

PyTorch：灶台。 它负责把食材（张量）加热、翻炒（计算），并告诉你哪一步出了问题（自动微分/调试）。学深度学习，先学它。

Hugging Face：半成品食材和菜谱库。 里面有很多预训练模型、数据集、微调工具。你不需要从种菜开始，拿半成品再加工就行。

DeepSpeed / Megatron：中央厨房。 一个灶台放不下大锅，就把切菜、炒菜、装盘分给很多人，还要保证大家配合。它们解决“多卡、多机、显存不够”的问题。

vLLM：出餐窗口。 菜做好了，要快速、稳定地端给很多客人。vLLM 通过批处理、KV cache 管理，提高吞吐、降低时延。**KV cache 就是把已经算过的历史信息存起来，避免每次从头算**；PagedAttention 则是它管理这份“记忆”的方法，像把记忆分页存放。

RAG：外挂资料库。 厨师记不住所有菜谱，就在旁边放一个随时能查的柜子。模型不知道的知识，先查再答。

W&B / MLflow：实验记录本。 今天试了哪个配方、火候多大、味道如何，都要记下来；否则一周后你也不知道哪次最好。

5.2 一个重点工具的“五联关系”：Transformer

它能做什么？ 翻译、摘要、问答、对话、代码生成；也能把图像切成 patch、把语音切成 token 来处理。

背后是什么概念？ 注意力（看哪里）、位置编码（谁在前谁在后）、残差（信息别丢）、LayerNorm（训练更稳）、下一 token 预测（怎么学）。

用之前要知道什么？ 线性代数（矩阵乘法）、概率（softmax 就是把一组分数变成概率）、反向传播、PyTorch 的基本操作。

对应浙大哪些课？ 人工智能引论 → 深度学习导论 → 高级机器学习；再往上，自然语言处理/计算机视觉导论会用到它。想深入实现，可以看 Stanford CS224N/CS336 作为拓展。

什么时候别用它？ 数据很少、任务很简单、边缘设备算力有限时，CNN/树模型可能更合适；超长文本、流式场景，可以关注 SSM/Mamba 等替代路线。

5.3 工具学习顺序

先 **PyTorch**，再 **Hugging Face**，然后 **一种训练加速工具**（FSDP/DeepSpeed），再 **一种推理工具**（vLLM），最后 **应用编排**（LangChain/LlamaIndex）。不要一上来同时学五个框架。

工具不是目的。 工具会更新，今天学的按钮明天可能改名；但“数据要干净、评测要独立、训练要能复现、服务要能回滚”这些原则不会过时。先学能让你把一条竖切走通的工具，再学“更高级”的工具。

06 / 数据、显卡与评测：三个最容易踩坑的地方

6.1 数据：食材不干净，后面全白费

模型从数据里学规律。数据有重复，模型会“背答案”；有泄漏，评测会虚高；有版权/隐私问题，项目可能直接停掉。

所以每份数据都要有一张“数据卡”：从哪来、什么许可、多大、什么语言/领域、怎么清洗、怎么切分、有没有个人信息、版本是什么。

6.2 显卡与显存：案板有多大，决定你能做多少菜

显存就像厨房的案板。模型权重、梯度、优化器状态、激活值都要放在上面。7B 模型全参训练，光优化器状态就可能超过 100GB；一张 24GB 显卡通常只能做 7B 的 QLoRA 微调，或者 1B 以下小模型训练。

所以“我要训一个 70B”不是一句口号，先算显存、算力、时间和预算。算力粗算可以用“参数量 × token 数 × 6”估算训练 FLOPs，再除以实际算力；**FLOPs 就是计算量，像“一共要做多少道算术题”**。真实预算至少留 2—3 倍余量。

6.3 评测：不能只让厨师自己说好吃

公开基准（MMLU、GSM8K、HumanEval）像模拟考，能看大概水平，但可能泄题、押题、和你的真实任务脱节。真正要看的，是三层：

- **能力层**：公开基准，看看通用能力。
- **任务层**：你自己的 100—1000 条测试集，看真实任务。
- **体验层**：人类评估，看有用、可信、可纠正。

> **可复现**
>
> 每次实验记录：代码版本、数据版本、随机种子、超参、硬件、评测脚本。没有这些，别人无法相信你的“提升”。

一张数据卡长什么样？ 至少写清楚：来源（网站/合同）、许可（能不能商用）、规模（文档/样本/token 数）、时间（知识截止）、处理（清洗/去重/去污染）、切分（训练/验证/测试）、隐私（有没有个人信息）、版本（哈希/日期）。这张卡不写，后面一定有人问“数据哪来的”。

07 / 浙大 2026 级人工智能专业：课程怎么串起来

这一节按你提供的《2026 级人工智能专业培养方案》[Z01] 整理。课程名来自方案；具体选课/学期以教务系统为准。

浙大的培养方案不是“一堆课”，而是一条链：**数理基础 → 编程与系统 → AI 核心 → 方向课 → 科研实践**。下面只列和“模型构建”最相关的课。

阶段	课程	在模型构建里干什么	标签
大一	微积分（甲）I/II、线性代数（甲）	看懂损失、梯度、矩阵运算	前置基础
大一	智能程序设计、人工智能前沿与创新（新生研讨课）	会写代码；建立领域全景	前置基础
大二	离散数学及其应用、高级数据结构与算法分析	复杂度、图、搜索、工程基础	前置基础
大二	概率论与数理统计	分布、估计、评测置信	前置基础
大二	人工智能引论、深度学习导论	建立 AI/深度学习主框架	核心覆盖
大二	计算机系统 I/II、人工智能芯片与系统	内存、并行、推理部署	部分涉及
大二	人工智能前沿实践	把知识变成动手经验	实践应用
大三	优化基本理论与方法、高级机器学习	训练为什么收敛、模型怎么选	核心覆盖
大三	因果推理、人工智能逻辑、认知神经科学导论	因果、知识表示、认知启发	部分涉及
大三	计算机视觉导论、自然语言处理	两大应用方向	部分涉及
大三	人工智能伦理与安全	偏见、隐私、安全、责任	部分涉及
大三	科研实践 I/II	进课题组、做小研究	实践应用
大四	人工智能科研实践、科研实践 III—VI、毕业论文（设计）	完整研究/工程项目	实践应用
大四	智能理论/智能模型/智能系统/智能工程方向课（含数据挖掘与表征学习、大模型基础等）	按兴趣深入	自主拓展

不要按课名猜内容

“机器学习”和“高级机器学习”可能完全不同的深度；“人工智能引论”在不同年份也会调整。选课前看课程大纲、问任课老师；不确定的标“待核实”。

7.1 一条“最小路线”

如果你只想先入门，不要一次选满方向课。按这个顺序：

1. **大一**：微积分、线代、智能程序设计；把 Python 和矩阵运算用熟。
2. **大二**：概率统计、人工智能引论、深度学习导论；手写一个小 MLP，跑通训练/评测。
3. **大三**：高级机器学习 + 一门方向课（NLP 或 CV）+ 科研实践；做一个完整项目。
4. **大四**：科研实践/毕设；把项目做成可复现、可展示的作品。

08 / 培养方案怎么变成能力：三条路径

浙大培养方案有 8 条毕业要求。这里只取和模型构建最相关的 5 条，拆成三条路径。

培养要求（浙大）	具体能力	课程	工具	后续实践
数理与科学基础	能读公式、能设计实验	微积分、线代、概率统计、大学物理	数学推导、统计分析	数据分析小项目
掌握 AI/ML/CV/NLP 基本理论	能解释模型为什么有效/失效	人工智能引论、深度学习导论、高级机器学习、CV/NLP	PyTorch、Transformer	复现一个小模型
理解、分析、设计、开发	能把模型做成系统	计算机系统、人工智能芯片与系统、科研实践	FSDP/DeepSpeed、vLLM	RAG/微调服务
科研与工程实践、创新	能做研究、能写代码、能分析	科研实践 I—VI、人工智能科研实践、毕设	W&B、实验设计	论文复现/开源项目
团队/表达/伦理	能讲清、能负责	人工智能伦理与安全、科研实践	模型卡、红队、文档	答辩/项目评审

8.1 三条路径，不是三个“人设”

科研型：把“为什么有效”问到底。多选高级机器学习、方向课、科研实践；目标是能复现论文、能做消融、能写清楚失败原因。

工程型：把“怎么稳定跑起来”问到底。多选计算机系统、人工智能芯片与系统、科研实践；目标是能训练、能部署、能压测、能回滚。

设计/产品型：把“人怎么用”问到底。多选人工智能伦理与安全、认知神经科学导论、智能产品设计（方向课）；目标是能访谈、能做原型、能评估体验。

三个作品，证明三种能力： 一个“经典基线”项目，证明你会做 baseline、懂评估；一个“微调/从零小模型”项目，证明你理解模型内部；一个“RAG/领域助手”项目，证明你能把模型变成系统。每个项目用“问题—约束—方案—评测—失败—反思”六段式讲清楚，比“我用了什么框架”更有说服力。

> **课程不是目的**
>
> 课程给你工具和语言，项目给你手感和判断。每学一门课，至少做一个小项目；否则“学过”很快会忘。

09 / 我现在的理解到哪：五级量表与自评

这一节用“学骑自行车”来理解五级。

层级	名称	用骑车打比方	证据
L0	未接触	没碰过自行车	说不出基本部件
L1	能复述	知道有车把、刹车、踏板	能解释概念、看懂教程
L2	能解释	知道为什么转弯要倾斜、刹车不能只捏前轮	能说明“为什么/何时失败”
L3	能复现	能独立骑上路，能处理小坡	独立复现最小例子/实验
L4	能改造	能换零件、越野、载人	针对新任务改结构/目标/系统
L5	能创新/迁移	能设计一辆新车，或教别人	提出新方法/迁移到新领域

9.1 十个关键节点（请团队按证据填写）

关键节点	建议目标	资料可支撑	当前（L0—L5）	证据/下一步
任务形式化与验收	L3	L3		写任务卡；找 1 个反例
数据清洗/切分	L3	L3		处理真实语料；报告重复率
Token/嵌入	L3	L3		比较中英文 token；做相似度
损失/梯度	L3	L3		从零写训练循环；过拟合小 batch
注意力/Transformer	L4	L3		手写 attention；训练小模型
预训练/微调	L3	L2		读一篇论文；跑一次微调
LoRA	L3	L3		单卡微调 7B/8B；对比全参
对齐	L2	L2		读 DPO/RLHF；跑小型 DPO
RAG/评测	L3	L3		建领域问答；评估引用
部署/复现	L3	L3		部署 API；写模型卡

怎么填

不要写“熟练掌握”。每个层级都要有证据：代码、报告、项目链接。团队交叉质疑，取“最低证据”。

10 / 十二周行动与主要来源

10.1 十二周（每周 8—10 小时）

阶段	周	目标	交付
打地基	1—2	任务卡、基线、50 条评测集、数据卡	任务卡+数据卡+评测集 v0
跑通	3—4	端到端 Demo、100 条评测、错误分类	可运行系统+评测报告 v1
改进	5—8	只改一个变量；消融；稳健性	对比实验+消融表
交付	9—12	服务化、压测、红队、模型卡、复现包	API+报告+复现包

10.2 最终交付清单

- ☐ 代码可运行；☐ 数据卡/版本/切分；☐ 模型卡；☐ 评测集与错误分析；☐ 服务/演示；☐ README+复现包；☐ 红队/许可/回滚。

10.3 主要来源

[Z01] 浙江大学. 2026 级人工智能专业培养方案. 浙江大学（用户提供的 PDF）, 2026.

[C16] 清华大学. 清华大学人工智能班（智班）本科培养方案. 清华大学本科招生/培养方案公开文件, 2022. https://www.tsinghua.edu.cn/jxjywj/bkzy2022/zxzy/29-3.pdf

[C01] Stanford University. CS336: Language Modeling from Scratch (Spring 2026). 课程网站, 2026. https://cs336.stanford.edu/

[C02] Stanford University. CS224N: Natural Language Processing with Deep Learning (Winter 2026). 课程网站, 2026. https://web.stanford.edu/class/cs224n/

[C07] MIT. 6.S191: Introduction to Deep Learning (2026). 课程网站, 2026. https://introtodeeplearning.com/

[C08] MIT EECS. 6.7960: Deep Learning (Fall 2024). 课程网站, 2024. https://phillipi.github.io/6.7960/

[C13] UC Berkeley EECS. CS 189: Introduction to Machine Learning. 课程目录, 2026. https://www2.eecs.berkeley.edu/Courses/CS189/

[P01] Vaswani, A. et al.. Attention Is All You Need. NeurIPS 2017, 2017. https://arxiv.org/abs/1706.03762

[P05] Hoffmann, J. et al.. Training Compute-Optimal Large Language Models. NeurIPS 2022, 2022. https://arxiv.org/abs/2203.15556

[P06] Hu, E. J. et al.. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022, 2022. https://arxiv.org/abs/2106.09685

[P08] Rafailov, R. et al.. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023, 2023. https://arxiv.org/abs/2305.18290

[P23] Lewis, P. et al.. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020, 2020. https://arxiv.org/abs/2005.11401

[P26] Liang, P. et al.. Holistic Evaluation of Language Models (HELM). TMLR 2023, 2022. https://arxiv.org/abs/2211.09110

[P27] Hendrycks, D. et al.. Measuring Massive Multitask Language Understanding (MMLU). ICLR 2021, 2021. https://arxiv.org/abs/2009.03300

[P46] Mitchell, M. et al.. Model Cards for Model Reporting. FAT* 2019, 2019. https://arxiv.org/abs/1810.03993

[S01] PyTorch. PyTorch Documentation. 官方文档, 2026. https://pytorch.org/docs/stable/index.html

[S02] Hugging Face. Transformers Documentation. 官方文档, 2026. https://huggingface.co/docs/transformers/index

[S06] Microsoft. DeepSpeed. 官方文档, 2026. https://www.deepspeed.ai/

[S08] vLLM Project. vLLM Documentation. 官方文档, 2026. https://docs.vllm.ai/

[D03] EleutherAI. The Pile. 官方网站, 2020. https://pile.eleuther.ai/

[D06] Stanford Vision Lab et al.. ImageNet. 官方网站, 2026. https://www.image-net.org/

[D13] LMArena. Chatbot Arena. 官方网站, 2026. https://lmarena.ai/

[G01] NIST. AI Risk Management Framework (AI RMF 1.0). NIST, 2023. https://www.nist.gov/itl/ai-risk-management-framework

完整来源、A/B/C 证据分级与访问日期见长版附录 C。